TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

Scientific Problem-Solving Using Decision Making and Looping

Programs that choose and programs that repeat: if–else ladders and switch for classification problems, for/while loops for accumulations — the two control structures behind nearly every scientific calculation.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 5 · LEVEL 2 TERM 1

1. Aim

To solve typical scientific/engineering problems in C using decision statements (if, if–else, else-if ladder, switch) and loops (for, while, do–while), verifying every result by hand.

2. Concepts you need

- **Relational and logical operators:** == != < > <= >= compare; && (AND), || (OR), ! (NOT) combine conditions. Remember: == compares, = assigns — confusing them compiles silently and decides wrongly.
- **if / else-if ladder:** conditions are tested top to bottom; the first true one runs and the rest are skipped — order the ladder from the most specific/highest band downward.
- **switch:** chooses among constant cases of an int/char value; every case needs a **break**, or execution falls through into the next case.
- **for loop:** for (initialise; condition; update) — use when the repetition count is known. **while:** tests first, may run zero times. **do–while:** tests last, always runs at least once (Experiment 3's pre-test/post-test distinction, in code).
- **Accumulator pattern:** initialise before the loop (sum = 0, fact = 1 — note the different starting values), update inside, use after. **break** leaves a loop at once; **continue** skips to the next repetition.

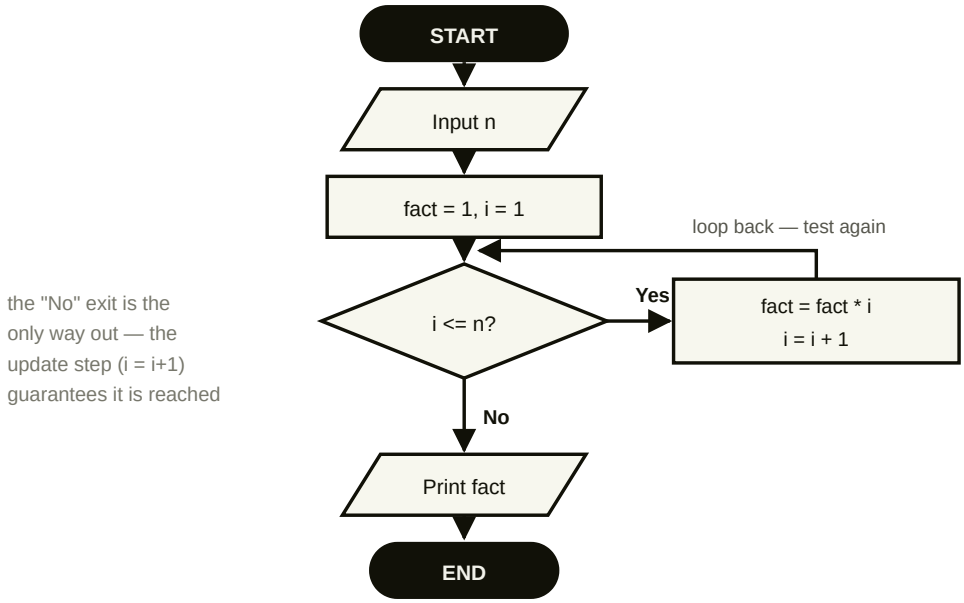


Figure 1 — Flowchart of the factorial program (Worked Example C). A loop in a flowchart is a decision whose Yes-branch returns; remove the update step and the loop never ends — the commonest beginner fault.

3. Lab procedure

1. For each program: predict the output for the given test data *before* running, trace on paper, then run and reconcile any difference.
2. Test each decision program at its **boundaries** — for the grade ladder: 80, 79, 40, 39 — not only at comfortable middle values.
3. Deliberately delete the loop update once (in a copy) and observe the infinite loop; stop it with Ctrl+C. Recognising a hung loop's behaviour is part of the experiment.

4. Worked examples

A — Decision: grade from marks (else-if ladder)

```
#include <stdio.h>

int main(void)
{
    int marks;
    printf("Enter marks: ");
    scanf("%d", &marks);
    if (marks >= 80)
        printf("Grade: A+\n");
    else if (marks >= 70)
        printf("Grade: A\n");
```

```
    else if (marks >= 60)
        printf("Grade: B\n");
    else if (marks >= 50)
        printf("Grade: C\n");
    else if (marks >= 40)
        printf("Grade: D\n");
    else
        printf("Grade: F\n");
    return 0;
}
```

Output (input 75): the first test (75 ≥ 80) fails, the second (75 ≥ 70) succeeds:

```
Enter marks: 75
Grade: A
```

B — Loop as accumulator: sum of even numbers 1–20

```
#include <stdio.h>

int main(void)
{
    int i, sum = 0;
    for (i = 2; i <= 20; i = i + 2)
        sum = sum + i;
    printf("Sum = %d\n", sum);
    return 0;
}
```

Output:

```
Sum = 110
```

Hand check: 2 + 4 + … + 20 = 2 × (1 + 2 + … + 10) = 2 × 55 = **110**. ✓

C — Loop as product: factorial (Figure 1)

```
#include <stdio.h>

int main(void)
{
    int i, n;
    long fact = 1;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        fact = fact * i;
    printf("Factorial of %d = %ld\n", n, fact);
```

```
    return 0;
}
```

Output (input 6):

```
Enter n: 6
Factorial of 6 = 720
```

Hand check: fact runs 1 → 1 → 2 → 6 → 24 → 120 → 720 as i runs 1…6. ✓ fact is declared `long` because factorials outgrow int quickly (13! already exceeds a 32-bit int) — type choice is part of problem solving.

5. Practice tasks

1. Write a program that reads an integer and reports whether it is prime. Give the traced verdicts for 29 and for 21, and state the loop's test range and why testing beyond √n is wasted work.
2. Write a program that reverses the digits of an integer (input 1234). Give the output and a step trace of the loop.
3. Write a program that prints the sum of the digits of an integer (input 482). Give the output and trace.

Answer key

1.
```
#include <stdio.h>

int main(void)
{
    int n, i, flag = 1;
    printf("Enter n: ");
    scanf("%d", &n);
    if (n < 2)
        flag = 0;
    for (i = 2; i * i <= n; i++)
    {
        if (n % i == 0)
        {
            flag = 0;
            break;
        }
    }
    if (flag)
        printf("%d is prime\n", n);
    else
        printf("%d is not prime\n", n);
    return 0;
}
```

29: tests i = 2, 3, 4, 5 (i×i ≤ 29); remainders 1, 2, 1, 4 — none zero → **29 is prime**. 21: i = 2 leaves 1, i = 3 divides exactly → flag = 0, break → **21 is not prime**. Beyond √n any factor must pair with one already tested below √n, so the search can stop there.

2.
```c
#include <stdio.h>

int main(void)
{
    int n, rev = 0, d;
    printf("Enter an integer: ");
    scanf("%d", &n);
    while (n > 0)
    {
        d = n % 10;
        rev = rev * 10 + d;
        n = n / 10;
    }
    printf("Reversed = %d\n", rev);
    return 0;
}
```

Trace for 1234: d=4, rev=4, n=123 → d=3, rev=43, n=12 → d=2, rev=432, n=1 → d=1, rev=4321, n=0 → loop ends. Output: **Reversed = 4321**.

3.
```c
#include <stdio.h>

int main(void)
{
    int n, sum = 0;
    printf("Enter an integer: ");
    scanf("%d", &n);
    while (n > 0)
    {
        sum = sum + n % 10;
        n = n / 10;
    }
    printf("Sum of digits = %d\n", sum);
    return 0;
}
```

Trace for 482: sum = 2, n = 48 → sum = 2+8 = 10, n = 4 → sum = 14, n = 0. Output: **Sum of digits = 14**.
